Published: 06 March 2021

Citation in BibTeX format

SSIP 2020: 2020 3rd International Conference on Sensors, Signal and Image Processing
October 9 - 11, 2020
Prague, Czech Republic

RESEARCH-ARTICLE

Generating Test Scripts for Web-Based Applications

PANTAKARN SRIWICHAINAN, Chulalongkorn University, Bangkok, Thailand

TARATIP SUWANNASART, Chulalongkorn University, Bangkok, Thailand

SSIP '20: Proceedings of the 2020 3rd International Conference on Sensors, Signal and Image Processing (October 2020)
https://doi.org/10.1145/3441233.3441248
ISBN: 9781450388283

Generating Test Scripts for Web-Based Applications

Pantakarn Sriwichainan
Department of Computer Engineering, Chulalongkorn University, Bangkok, Thailand

Taratip Suwannasart
Department of Computer Engineering, Chulalongkorn University, Bangkok, Thailand

ABSTRACT

Testing process is an essential part of the software development process as it reveals faults of the system which leads to software reliability improvement. Manual software manual testing is a process which testers discover software's faults by hands. Therefore, any change on the software requires tremendous effort and time to execute test cases such as regression tests. Even though we can utilize automation testing tools to speed up the execution of test cases, experienced testers, who are equipped with automation testing knowledge, are inevitably required. Since novice testers usually consume a significant amount of time to develop test scripts, this paper aims to establish an approach to simplify test scripts generation for web-based applications using URL and XSD as inputs. Our approach elicits input elements from a web page using a provided URL and then analyzes their values using XSD to create test data using boundary value testing technique. This produces test scripts that run under Robot framework.

CCS CONCEPTS

• **Software and its engineering**; • **Software creation and management**; • **Software verification and validation**; • **Empirical software validation**;

KEYWORDS

Software Testing, Automation Testing, Test Script, Test Automation Framework, Robot Framework, Selenium Boundary Value Testing

ACM Reference Format:
Pantakarn Sriwichainan and Taratip Suwannasart. 2020. Generating Test Scripts for Web-Based Applications. In *2020 3rd International Conference on Sensors, Signal and Image Processing (SSIP 2020), October 09–11, 2020, Prague, Czech Republic.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3441233.3441248

1 INTRODUCTION

Software testing plays an important role in software development as it unveils faults, verifies correctness, assures quality, and ensures reliability of the software system [1]. Testers perform manual testing to verify and prevent fault [2], but this generally requires a large amount of time and efforts [3].

Using test automation can reduce time [4] [5] since test scripts are created to automatically verify the system whether it works as expected or not. A test script is a set of sequential actions performed on a system under test. However, creating a test script is a daunting task because it demands programming skill and a deep understanding of the automation framework [6] [7]. Nowadays, automation framework [8] becomes a major component in automation testing. Robot framework, one of the widespread automation frameworks, helps testers develop test scripts. Nonetheless, its learning curve is rather steep at the beginning.

The prior study "Design and Implementation of Test Case for Automated Testing Using UI Structure" [9] provides a framework called "Neo Automation Framework" for developing automated test cases. The framework generates a UI structure of the software under test and then uses it to automatically generate a test case. However, the framework allows only partial modification of the test case and works exclusively for .NET applications.

The purpose of this paper is to describe a concept of generating test scripts for web-based applications using an automation framework. Our concept uses HTML [10], and XSD [11] as input information to generate test scripts. First, HTML structure is analyzed to extract input elements which are then used to generate test scripts with no test data. Next, XSD is analyzed to extract boundary value of inputs of the web-based applications to generate test data. Finally, the test scripts are combined with test data to generate complete test scripts.

2 BACKGROUND AND RELATED WORK

2.1 Keyword-driven testing

Keyword-Driven [12] is one of automated testing methodologies. Keywords are the fundamental part of test script implementation. A keyword is an action that can be executed on the web-based applications. The elements of Keyword-Driven are keywords, object identifier, and parameters. Separating keywords from the program makes it easier to maintain. Another advantage of this method is no that programming knowledge is required since it uses a natural language.

Keywords: A creation keyword is a simple phrase used to command and control objects in the program.

Object Identifier: It is used to define a specific object's position in the program.

Parameters: A keyword may require parameters to specify the input data which depends on its associated parameters.

For example, "Input Text id=Firstname John". and "Click Button id=submit". The first example, which keyword is Input Text with "Firstname". as the object identifier, requires an input parameter with value "John"., while the second example, the "Click Button". keyword, requires no parameter, but just only the object identifier—"submit". As shown in Table 1

SSIP 2020, October 09–11, 2020, Prague, Czech Republic

ACM ISBN 978-1-4503-8828-3/20/10…$15.00
https://doi.org/10.1145/3441233.3441248

Table 1: Examples of using keywords

Step	Keyword	Object Identifier	Parameter
1	Input Text	id="Firstname"	john
2	Input Text	id="Lastname"	canon
3	Click Button	id="Submit"	-

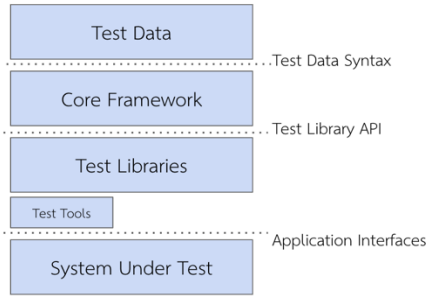

Figure 1: Robot framework architecture.

2.2 Robot framework

Robot Framework [13] is a test automation framework based on a theoretical Keyword-Driven Testing approach, which defines keywords for designing test scripts. So, a test script is a set of keywords that performs on a system under test. Figure 1 shows the architecture layer of Robot Framework.

Test Data: Test data is used as input data in a test script which can be defined as Settings, Variables, Test Cases, or Keywords sections of the script

Core Framework: Core Framework is an intermediate layer between Test Data layer and Test Libraries layer. Test reports are generated from this layer.

Test Libraries: Test Libraries are collections of keywords which we use to design test scripts. Robot Framework's test libraries can be divided into three types which are a standard library, an external library, and other Libraries.

- Standard Libraries: Standard Libraries are built-in with the Robot Framework. Keywords of these libraries can be used without additional installation.
- External Libraries: They are separately developed by well-known third parties. Since these libraries are not distributed with the framework's package, additional installation is required, for example, SeleniumLibrary.
- Other Libraries: As open-source software, Robot Framework allows users to implement their own libraries to integrate with the framework using any programming language. For example, users can create their test APIs using Python.

Test Tools: Test Tools are application interfaces that interact with system under test.

System Under Test: A specific target system that test scripts can be run on.

Table 2: Example of keywords defined by Selenium library

Keyword	Arguments	Documentation
Input text	locator, text, clear=True	The keyword Input text can identify the text field by locator. When clear is true, the element cleared old text before the new text inputted into the element. When clear is false, the element not cleared the old text.
Click Button	Locator	The keyword Click Button can click the button identify by locator.

Table 3: Input fields under element groups

Element groups	Input fields
Select element	*l* Drop-down list
Input element	*l* Date
	l Email
	l Tel
	l Number
	l Text
	l Radio
	l Checkbox
Button element	*l* Button
	l Reset
	l Submit

2.3 SeleniumLibrary

SeleniumLibrary [14] is one of Robot Framework's external test libraries used to test web-based applications. The general keywords defined in this library is used to control input elements on web-based applications. The elements of a SeleniumLibrary keyword is identical to those of a Keyword-Driven testing keyword, which also consists of a keyword, a locator, and parameters. Moreover, SeleniumLibrary provides good documentation that elaborates on how to use with its keywords. Table 2 displays example keywords provided by SeleniumLibrary document.

2.4 The element type

HTML elements [15] defined by W3C are objects that can be displayed on a web page, such as a drop-down list, a checkbox, or a button. These elements are divided into three groups which are select elements, input elements, and button elements. The elements used in this paper are shown in Table 3

2.5 Robust boundary value testing

Boundary Value Testing (BVT) is a software testing technique which primarily focuses on the boundary's maximum and minimum values. BVT are categorized into four types as follows.

- Normal BVT
- Robust boundary value testing (RBVT) [16]

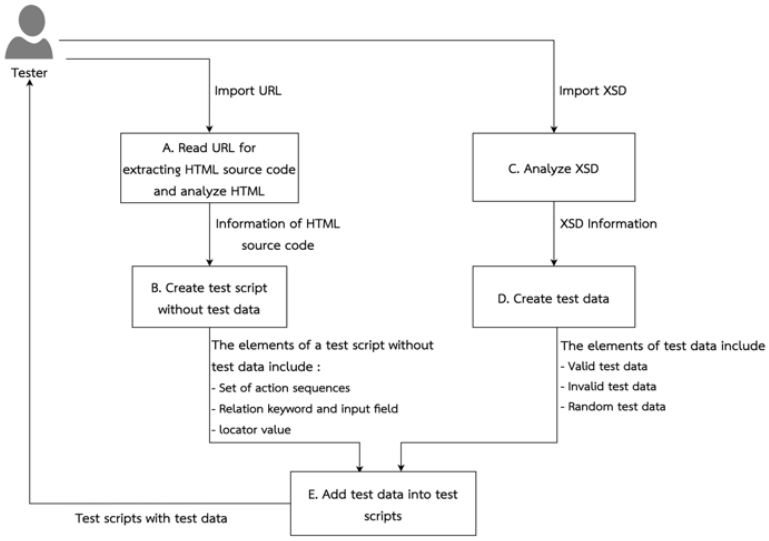

Figure 2: A conceptual process diagram.

- Worst-case BVT
- Robust worst-case BVT

The RBVT considers two input domains. One is values in-range or valid inputs which are min, min+, nom, max- and max. The other is, out-of-range or invalid inputs which are min- and max+.

3 PROPOSED APPROACH

To generate test scripts for web-based applications, we first read the given URL to pull out HTML source code then analyze HTML, and create test script without test data. Next, we analyze XSD to create test data. Lastly, we add the test data into the test script as shown in Figure 2

3.1 Read URL for extracting HTML source code and analyze HTML

HTML source code of the system under test is read from the given URL and analyzed to extract elements for composing test scripts. At this step, test data is not incorporated into the test scripts. Figure 3 displays a web-based application with a customer details form read from URL "http://localhost/php-details-form'', while Figure 4 depicts its HTML source code with input fields firstName, lastName, and age.

3.2 Create test script without test data

In this step, the HTML source code is further analyzed to create an order of input fields, from left to right and top to bottom. Figure 5 shows "First Name", "Last Name", and "Age" text boxes on the right-hand side which are corresponding to their HTML input elements with id "firstName", "lastName", and "age" on the left-hand side, consecutively.

Next, a relationship of each input field and SeleniumLibrary keyword is established. Table 4 displays relationships between input fields named "First Name", "Last Name", and "Age" and SeleniumLibrary keyword "Input Text".

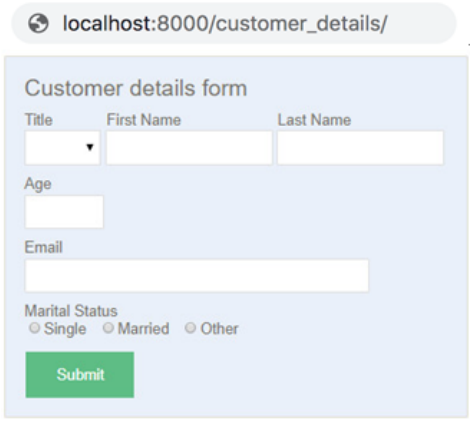

Figure 3: An example of URL and a web-based application.

```
▼<div class="input-row">
    <br>
  ▶<span class="pull-left">…</span>
    <br>
    <input type="text" class="input-field"
    name="firstName" id="firstName" size=
    "20">
    <input type="text" class="input-field"
    name="lastName" id="lastName">
  </div>
▼<div class="input-row">
    <label>Age</label>
    <span id="age-info" class="info"></span>
    <br>
    <input type="number" class="input-field"
    name="age" id="age">
  </div>
```

Figure 4: An example of HTML source code from URL.

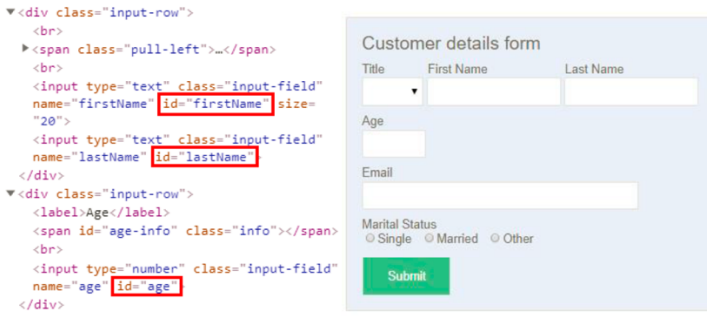

Figure 5: Tree locator examples from HTML source code

Table 4: The relation between input field and Selenium library keywords

Order No.	Input field		SeleniumLibrary keywords
	Name	Type	
1	First Name	Text	Input Text
2	Last Name	Text	Input Text
3	Age	Number	Input Text

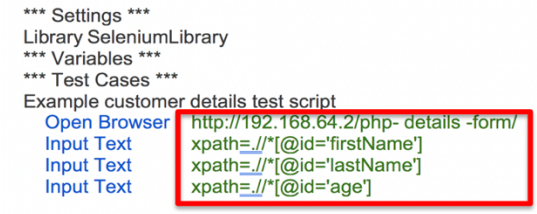

Figure 6: An example of a test script without test data.

A keyword's locator is then elicited from the input field's id. For example, the input fields with id "firstName", "lastName", and "age" enclosed as highlighted in Figure 5 are elicited and transformed into keyword's locators as highlighted in Figure 6. The order of input fields, relationships, and locators are used to generate a test script without test data.

3.3 Analyze XSD

The XSD file, which corresponds to the imported HTML input elements, is imported and analyzed. The type of a data set (either string or integer) is determined by the "restriction base" attribute. In addition, the boundary values are set by "minLength" and "maxLength" attributes. Figure 7 shows three elements of an example XSD file which are "firstName", "lastName", and "age". The first two elements share the same type which is string and the same boundary values with 5 and 15 as the minimum and the maximum, respectively. While, the last element is of integer type and ranges between 1 and 150 inclusively.

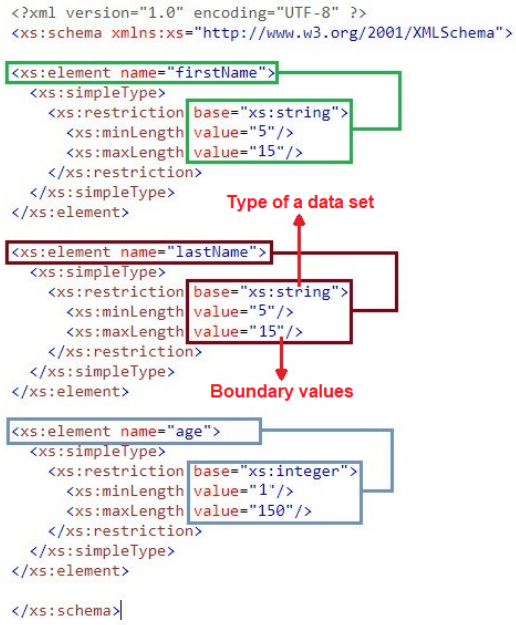

Figure 7: An example of eliciting boundary values in XSD.

Table 5: An example test data, valid and invalid values generated by RBVT

Test data	Input field FirstName	Input field LastName	Input field Age
Valid value			
1	johnjohn	canoncan	75
2	johnj	canon	1
3	johnjo	canonc	2
4	johnjohnjohnjo	canoncanoncano	149
5	johnjohnjohnjoh	canoncanoncanon	150
Invalid value			
1	john	cano	0
2	johnjohnjohnjohn	canoncanoncanonc	151

```
*** Settings ***
Library SeleniumLibrary
*** Variables ***
*** Test Cases ***
Example customer details test script
    Open Browser    http://192.168.64.2/php- details -form/    Chrome
    Input Text    xpath=.//*[@id='firstName']    johnjohn
    Input Text    xpath=.//*[@id='lastName']    canoncan
    Input Text    xpath=.//*[@id='age']    75
```

Figure 8: An example of test script.

3.4 Create test data

Depending on the type of input field, the method used to generate test data can be either Robust Boundary Value Testing (RBVT) method or randomization. If the type is numeric or string, the RBVT is used, otherwise, randomization. Generated in-range data are used to verify valid values. Likewise, the generated out-of-range data are used to verify invalid values. Table 5 displays seven test cases with values generated from the XSD file shown in Figure 7

Table 6: An example test script with valid and invalid values

	Keyword	Open Browser	Input Text	Input Text	Input Text
	Locator	http://192.168.64.2/php- details -form/	xpath=.//*[@id='firstName']	xpath=.//*[@id='lastName']	xpath=.//*[@id='age']
Test data	Valid data				
	1	Chrome	johnjohn	canonc	2
	2	Chrome	Johnj	canoncanon cano	149
	3	Chrome	johnjo	canoncanon canon	150
	4	Chrome	Johnjohnj ohnjo	canoncan	75
	5	Chrome	johnjohnj ohnjoh	canoncan	75
	Invalid data				
	6	Chrome	John	canoncan	75
	7	Chrome	johnjohnj ohnjohn	canoncan	75
	8	Chrome	johnjohn	cano	75
	9	Chrome	johnjohn	canoncanon canonc	75
	10	Chrome	johnjohn	canoncan	0
	11	Chrome	johnjohn	canoncan	151

3.5 Add test data into test scripts

Finally, test data generated in the previous step are merged into test scripts generated from step B shown in Figure 8

Since there is only one test case, "Example customer details test script in Figure 3", and test data are generated as shown in Table 5 consecutively, therefore, a total of eleven test script variants can be constructed for the three input fields–FirstName, LastName and Age, using RBVT single fault condition. These test script variants are displayed in Table 6

4 CONCLUSION

To summarize our proposed approach for generating test scripts for Web-Based applications, the given URL is read to collect the HTML input elements, and an XSD file is imported. After that, the input elements are analyzed to create test scripts without test data. Finally, the XSD is then analyzed to create test data before adding to the test script. With this, time consumption in creating test script for web-based applications can be significantly reduced. However, our approach can generate test scripts for single-page web-based applications with unique id or class for each input fields. Also, either id, class, or name of an input field must be the same as its corresponding element's name in the XSD. In addition, our test scripts can be executed by Robot Framework, yet relationships between input fields are not supported. However, we are planning to eliminate these limitations by generating test scripts that can work on multiple-page web-based applications.

REFERENCES

[1] International Standard ISO/IEC/IEEE 29119-1. Software and systems engineering — Software testing — Part 1: Concepts and definitions, ISO/IEC 2013, pp. 12-14.
[2] H. Itkonen ; M. V. Mäntylä and C. Lassenius, "Defect Detection Efficiency: Test Case Based vs. Exploratory Testing", First International Symposium on Empirical Software Engineering and Measurement (ESEM 2007), Spain, 2007.
[3] Myers, G.J., C. Sandler, and T. Badgett, The art of software testing. 2011: John Wiley & Sons. pp. 178-179.
[4] Axelrod, A., Complete Guide to Test Automation. 2018: Springer. pp. 13-15.
[5] International Software Testing Qualifications Board (ISTQB), Certified Tester Foundation Level Syllabus. 2018. pp. 65-66.
[6] International Software Testing Qualifications Board (ISTQB), Certified Tester Advanced Level Syllabus Test Automation Engineer 2016. p. 13.
[7] International Software Testing Qualifications Board (ISTQB), Certified Tester Advanced Level Syllabus Test Automation Engineer. 2016. p. 31.
[8] Palani, N., Software Automation Testing Secrets Revealed. 2016: Becomeshakespeare. com. p.4.
[9] Harnvorawong, N. and T. Suwannasart. Design and Implementation of Test Case for Automated Testing Using UI Structure. in International MultiConference of Engineers and Computer Scientists 2014 (IMECS 2014). 2014. Hongkong.
[10] W3C. HTML 5.2 Specification. [cited 2019 October, 22]; Available from: http://www.w3.org/TR/html52.
[11] W3C XML Schema Definition Language (XSD). [cited 2019 October, 22]; Available from: https://www.w3.org/TR/xmlschema11-1.
[12] Tailoring International Standard ISO/IEC/IEEE 29119-5. in Software and systems engineering Software testing Part 5: Keyword-Driven Testing. 2016. IEEE.
[13] Robot Framework Foundation. Robot Framework. [cited 2019 October, 15]; Available from: http://robotframework.org.
[14] Robot Framework Foundation. SeleniumLibrary. [cited 2019 October, 15]; Available from: https://robotframework.org/
[15] SeleniumLibrary/SeleniumLibrary.html.
[16] W3C. HTML 5.2 Forms. [cited 2020 February, 16]; Available from: https://www.w3.org/TR/html52/sec-forms.html#the-button-element.
[17] Jorgensen, P.C., Software testing: a craftsman's approach, ed. 4nd. 2014: CRC press. pp. 79-94.